

TakeLow - README

- 1 TakeLow
 - 1.1 Architecture
 - 1.1.1 Technology Stack
 - 1.1.2 Data Layer
 - 1.2 Enterprise RBAC (CASL)
 - 1.3 Quick Start
 - 1.4 Common Commands
 - 1.5 Seed Credentials
 - 1.5.1 Enterprise Roles
 - 1.6 Environment
 - 1.7 Testing & Verification
 - 1.8 Project Layout
 - 1.9 Database Migrations
 - 1.10 Key Features
 - 1.10.1 Core Platform
 - 1.10.2 Authentication & Authorization
 - 1.10.3 Payments & Settlement
 - 1.10.4 Admin & Analytics
 - 1.10.5 Performance & Scalability
 - 1.10.6 Security & Compliance
 - 1.10.7 Notifications
 - 1.11 Documentation
 - 1.12 Backup & Disaster Recovery

1 TakeLow

A mobile-first reverse-auction platform using a **Lowest Unique Bid (LUB)** mechanism. The winner is the user who submits the lowest unique bid amount before the timer expires.

“Take It. Low.”

1.1 Architecture

Three NestJS microservices + React web + React Native (Expo) mobile:

Service	Port	Responsibility
identity-service	3001	Better-auth authentication, user registration, wallet, PIN, OTP, notifications, RBAC, disputes, T&C
auction-engine	3002	Real-time bidding, WebSockets (Redis adapter), Redis bid tracking, BullMQ workers, winner determination, payments, product approval, payment reminders
query-service	3003	Read-optimized auction/product queries, Redis caching, favorites, admin stats, settlement reports, analytics, audit viewer, winner management
take-low-web	5173 (dev) / 80 (docker)	React + Tailwind SPA
take-low-app	Expo	React Native mobile app

1.1.1 Technology Stack

Layer	Technology
ORM	Prisma 7 (all 3 services)
Auth	Better-auth (identity-service) + JwtAuthGuard (auction-engine, query-service)
RBAC	CASL ability-based access control with enterprise role hierarchy
Database	PostgreSQL 15 + Citus 12.1 (distributed)
Cache	Redis 7 (caching, rate limiting, queues, WebSocket adapter, pub/sub)
Queue	BullMQ (batched bid persistence, notification dispatch)
Real-time	Socket.io 4 with Redis adapter for horizontal scaling
Framework	NestJS 10 + TypeScript 5 (strict mode)

Layer	Technology
Frontend	React 19 + Vite 5 + Tailwind CSS 3
Mobile	React Native 0.86 + Expo SDK 52
Deployment	Docker Compose (dev), Kubernetes (production)

1.1.2 Data Layer

PostgreSQL 15 (Citius) + Redis 7 (bid tracking, rate limiting, queues, caching, WebSocket adapter) + BullMQ (batched bid persistence).

1.2 Enterprise RBAC (CASL)

TakeLow implements enterprise-grade role-based access control using CASL:

Level 1: CEO / Admin	→ Full system access (all divisions, departments, sections)
Level 2: CXO	→ Division-level access (CF0, CC0, CT0)
Level 3: Director	→ Department-level access
Level 4: Manager	→ Section-level access
Level 5: Expert	→ Functional area access
Level 6: Specialist	→ Narrow module scope
Level 7: Analyst	→ Read-only/reporting access
Level 7: User	→ Standard user (bidding, wallet, favorites)

Features: - CASL ability factory with granular action+subject permissions -

@RequireAbility(action, subject) decorator for endpoint protection - Dynamic permission overrides (temporary elevated access with expiry) - Access decision audit logging (every access attempt logged) - Organizational structure (divisions → departments → sections) - UI enforcement API (GET /rbac/abilities returns permissions for frontend)

1.3 Quick Start

```
# One-time: install deps + start infra + migrate + seed
npm run setup

# Full stack (Docker -> backends -> web frontend)
npm run dev
```

```
# Mobile app (Expo)
```

```
npm run app
```

```
# Backends only (requires Docker for Postgres/Redis)
```

```
npm run services
```

1.4 Common Commands

Command	Description
<code>npm run docker:up</code>	Start Postgres + Redis containers
<code>npm run docker:down</code>	Stop containers
<code>npm run db:setup</code>	Docker -> wait -> migrate -> seed
<code>npm run db:reset</code>	Wipe volumes -> migrate -> seed
<code>npm run db:migrate</code>	Apply SQL migrations (tracked in <code>schema_migrations</code>)
<code>npm run seed</code>	Seed 30 users + enterprise roles + 24 products + 24 auctions

1.5 Seed Credentials

1.5.1 Enterprise Roles

Role	Phone	Password	Level
Admin/CEO	0911111111	1234	1
CXO (CTO)	0913320001	0000	2
CXO (COO)	0913320002	0000	2
Director	0913320003	0000	3
Manager	0913320006	0000	4
Expert	0913320010	0000	5
Specialist	0913320013	0000	6
Analyst	0913320016	0000	7

Role	Phone	Password	Level
User	0913320018 – 0913320028	0000	7

All users have T&C accepted (v1.0), wallet PIN set (0000 for users, 1234 for admins).

1.6 Environment

Each service has a `.env.example`. Copy to `.env` and fill in. Critical vars:

Variable	Description
<code>DATABASE_URL</code>	PostgreSQL connection string
<code>REDIS_URL</code>	Redis connection (standalone)
<code>REDIS_SENTINEL_HOSTS</code>	Redis Sentinel (HA) — comma-separated <code>host:port</code>
<code>REDIS_SENTINEL_MASTER_NAME</code>	Redis Sentinel master name
<code>REDIS_CLUSTER_NODES</code>	Redis Cluster (sharding) — comma-separated <code>host:port</code>
<code>JWT_SECRET</code>	JWT signing secret (must NOT be <code>take-low-jwt-secret</code>)
<code>JWT_REFRESH_SECRET</code>	JWT refresh token secret
<code>INTERNAL_API_KEY</code>	Service-to-service authentication
<code>CORS_ORIGINS</code>	Comma-separated allowed origins
<code>SIKINA_*</code>	SikinaPay payment gateway config
<code>AWASH_*</code>	Awash Bank payment gateway config
<code>SMS_API_KEY</code>	SMS Ethiopia API key

1.7 Testing & Verification

```
# Backend tests (per service)
cd identity-service && npm test      # 8 tests
cd auction-engine && npm test        # 54 tests
cd query-service && npm test         # 3 tests
```

```
# Type checking (all 5 projects)
npx tsc --noEmit

# Prisma commands (per service)
npm run prisma:generate           # Generate Prisma client
npm run prisma:migrate           # Create and apply migration
npm run prisma:deploy            # Deploy migrations (production)
npm run prisma:studio            # Open Prisma Studio GUI
```

1.8 Project Layout

auction-engine/	NestJS bidding + winner + payments + product approval
identity-service/	NestJS auth (Better-auth) + wallet + RBAC (CASL) + disputes
query-service/	NestJS read APIs + caching + analytics + settlement + audit
takeLow-web/	React + Vite + Tailwind
takeLow-app/	React Native (Expo)
database/migrations/	40 raw SQL migrations (idempotent, tracked)
k8s/	Kubernetes manifests + backup CronJobs
scripts/	dev/seed/migrate/backup/restore helpers
load-tests/	k6 bid-flood scenarios
docs/	Enterprise documentation (SRS, SDD, TTD/LLD, User Guide)

1.9 Database Migrations

40 idempotent SQL migrations tracked in `schema_migrations` :

Range	Description
000–033	Core schema (users, products, auctions, bids, payments, winners, audit logs, permissions)
034	Citus distribution for production (auto-skips on local dev)
035	T&C acceptance columns
036	Product approval workflow columns
037	T&C version tracking
038	Dispute resolution table
039	RBAC role hierarchy (divisions, departments, sections, permission overrides, access decisions)

1.10 Key Features

1.10.1 Core Platform

- **Lowest Unique Bid (LUB)** auction mechanism with Redis ZSETs
- **Real-time bidding** via Socket.io with Redis adapter for horizontal scaling
- **AES-256-GCM encryption** for bid amounts at rest (closed auctions)
- **BullMQ batch persistence** for high-throughput bid processing
- **Fair-play extensions** (24h extension if min bids not met or no unique bids)

1.10.2 Authentication & Authorization

- **Better-auth** with Prisma adapter (identity-service)
- **JwtAuthGuard** for token verification (auction-engine, query-service)
- **CASL RBAC** with 9 enterprise roles and granular permissions
- **Dynamic permission overrides** with temporary elevated access
- **Wallet PIN** with 30-minute lockout after 5 failed attempts
- **T&C acceptance** required before bidding (with version tracking)

1.10.3 Payments & Settlement

- **SikinaPay** and **Awash Bank** payment gateway integration
- **Internal wallet** payments with atomic fee deduction
- **Payment reminder cron** (every 30 minutes) via SMS + push
- **Settlement reports** with revenue, platform share, tax, commission
- **Winner rotation** for unpaid winners (24h deadline)

1.10.4 Admin & Analytics

- **Advanced analytics dashboard** (revenue, user, auction, bid, payment analytics)
- **Audit log viewer** with filtering, pagination, and CSV export
- **Winner management** (extend deadline, confirm payment, trigger rotation)
- **Product approval workflow** (PENDING → APPROVED → REJECTED)
- **Dispute resolution** system with admin review
- **CSV export** for users, transactions, auctions, products, settlements, audit logs

1.10.5 Performance & Scalability

- **Redis caching layer** for read-heavy endpoints (auction listings, products, stats)
- **Socket.io Redis adapter** for WebSocket horizontal scaling
- **Redis Sentinel/Cluster** support for high availability

- **Citus distributed PostgreSQL** for horizontal database scaling
- **Kubernetes HPA** for auto-scaling backend services

1.10.6 Security & Compliance

- **Immutable audit logs** for all admin actions, bidding events, payments, and access decisions
- **HMAC-SHA256** webhook signature verification with timing-safe comparison
- **Nonce validation** for replay attack prevention
- **Rate limiting** on login (5/min), bidding (10/s), OTP (3/min)
- **Backup & disaster recovery** with pgBackRest and WAL-G
- **RTO < 4 hours, RPO < 24 hours** with point-in-time recovery

1.10.7 Notifications

- **Push notifications** via Expo Push Service
- **SMS notifications** via SMS Ethiopia
- **In-app notifications** with bulk mark-read
- **Payment reminders** via SMS + push (6h and 1h before deadline)
- **Auction ending soon** alerts (5 minutes before)

1.11 Documentation

Enterprise-grade documentation in `docs/` :

Document	Version	Description
<code>SRS.md</code>	v6.0	Software Requirements Specification (IEEE-830 compliant)
<code>SDD.md</code>	v3.0	System Design Document (architecture, ADRs, capacity planning)
<code>TTD_LLD.md</code>	v3.0	Technical Specifications & Low-Level Design
<code>UserGuide.md</code>	v3.0	User Guide (user rights, dispute resolution, fair play)
<code>REDIS_HA.md</code>	v1.0	Redis High Availability Configuration

Document	Version	Description
PRISMA_BETTER_AUTH_MIGRATION.md	v1.0	Prisma + Better-auth Migration Guide

Generated DOCX and HTML versions in docs/output/ .

1.12 Backup & Disaster Recovery

```
# pgBackRest backups
bash scripts/backup-pgbackrest.sh full      # Full backup
bash scripts/backup-pgbackrest.sh diff     # Differential backup
bash scripts/backup-pgbackrest.sh verify   # Verify backup

# WAL-G cloud backups
bash scripts/backup-walg.sh push           # Push base backup to S3
bash scripts/backup-walg.sh list          # List available backups

# Disaster recovery restore
bash scripts/restore-disaster-recovery.sh restore      # Full restore
bash scripts/restore-disaster-recovery.sh pitr '2026-08-18 10:00:00' # Point-in-time recovery
```

Kubernetes backup CronJobs in k8s/backup-cronjob.yaml : - Daily full backup at 02:00 UTC
- Differential backup every 6 hours - WAL-G alternative for cloud storage

See docs/SRS.md for the full requirements specification, docs/SDD.md for the system design, SECURITY.md for the security model, and br.md for the business requirements status.